AI SOLUTION CATALOG

Multi-Agent AI Orchestrator



Smart AI helpers that build software faster, with fewer mistakes.

Fewer Bugs	6	5 min	Full Trace
Better Quality	Expert AI SKILLs	Setup Time	From Plan to Code and Testing

Confidential

SOLUTION OVERVIEW

Solution Name

Multi-Agent AI Orchestrator
(A Simple Prompt System for VS Code Copilot)

Category

Helps Developers Work Faster | Automates Daily Tasks | Understands Code Better

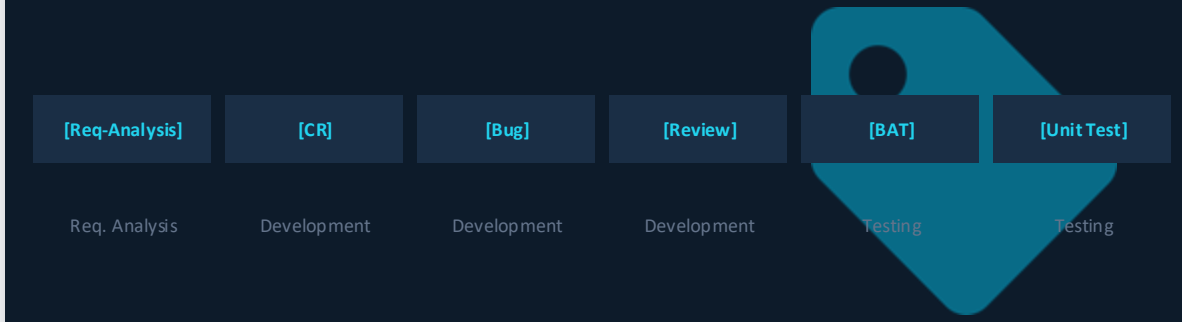
Who This Is For

- Software development teams & system integrators

In Simple Words

"AI often forgets things mid-task. Our system fixes that -- it sends every task straight to the right expert AI, every time."

6 Core Agents | 6 Action Tags
No Mixed-Up Tasks | Always Checks Before It Reads



PART 1 OF 3

Requirement Analysis



We turn the client's requirements into clear documents -- before writing any code.

SRS

Requirement Document

WBS

Task Breakdown

Skill File

Project Notes

3 Rounds

Double-Checked by 2 Agents
(Analyst & Reviewer)

Confidential

[Req-Analysis]

Requirement Analyst

1/5

/req-analysis

Overview: The [Req-Analysis] tag hires the Requirement Analyst -- a 3-agent pipeline (Maturity Assessor, Analyst, Reviewer) that turns raw requirement files into a clear SRS, a task list (WBS), and a reusable project skill file.

How It Works

1. Reads the attached files (Excel, Word, PDF, or image)
2. Analyst drafts the SRS; Reviewer checks it and fixes gaps
3. Maturity Assessor checks the requirements are clear enough
4. If anything is unclear, you get a maturity report to decide
5. Exports the SRS, WBS, and a reusable project skill file

Result: SRS + WBS + project skill file, ready to build from

Key Benefits

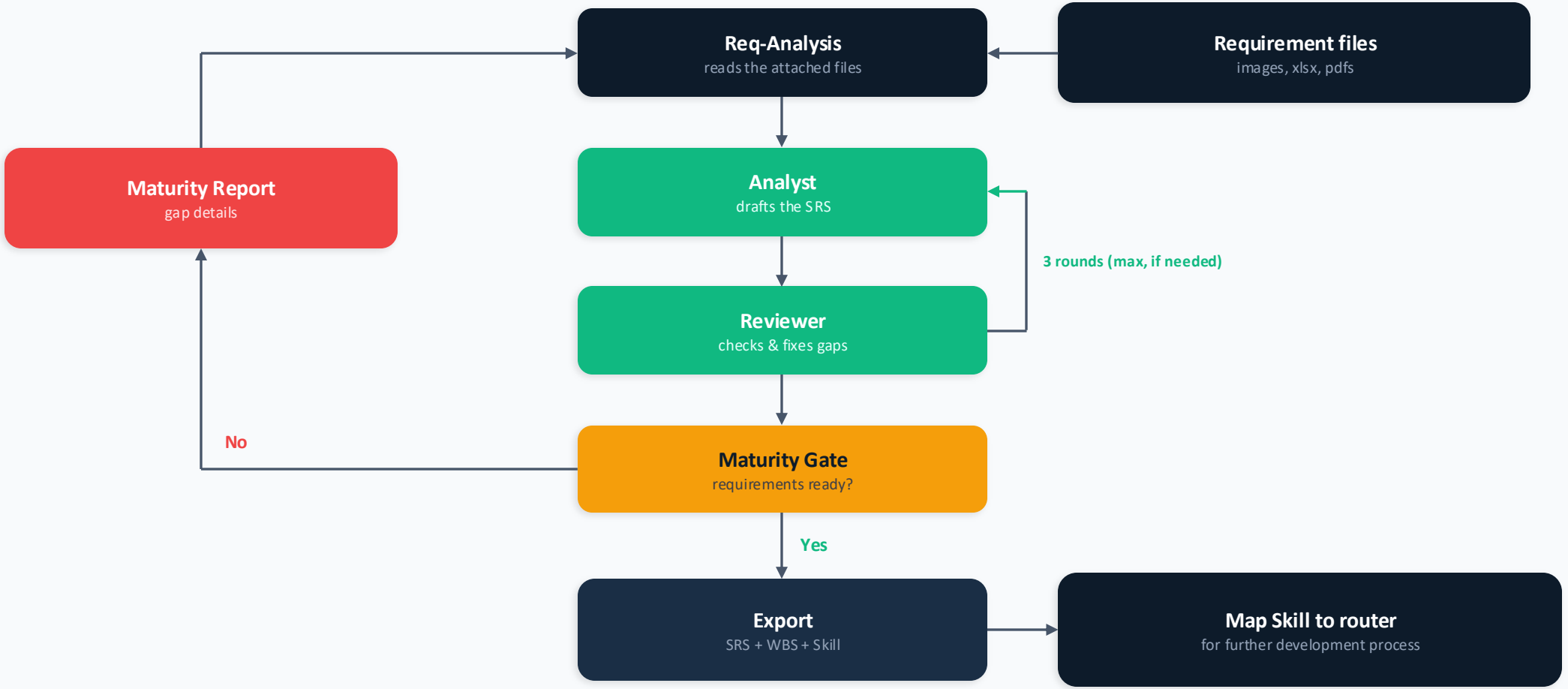
- Turns raw files into a ready SRS and WBS -- in one step
- A 3-agent pipeline double-checks the work, so nothing is missed
- A maturity gate flags unclear requirements before any SRS is written
- Can cross-check the requirements against the existing codebase
- The project skill file speeds up every future tagged prompt

[Req-Analysis]

Requirement Analysis -- Full Flow

flow `/req-analysis`

The full [Req-Analysis] flow: analyze, review, and gate on maturity -- looping back until the requirements are ready, then export.



[Req-Analysis]

Requirement Analyst -- How It Works

ex `/req-analysis`

One prompt turns a raw requirement file into a clear SRS and task list -- here is what that looks like.

What the Requirement Analyst Does

1. **You attach the requirement**
   An Excel, Word, PDF, or image file from the client.

2. **Maturity Assessor checks it**
   Is it clear enough to build from? If not, you get a gap report.

3. **Analyst + Reviewer write it**
   One drafts the SRS, the other checks it and fixes any gaps.

4. **It saves the documents**
   SRS, task list (WBS), and a reusable project skill file.

Example -- analyze a requirement doc

You type (and attach the file):

```
/req-analysis Analyze the attached
   requirement and produce SRS + WBS
```

produces ▾

`docs/srs-ceiling-height.md`
`docs/wbs/wbs-ceiling-height.md`
`.github/skills/.../ceiling-height.md`

```
SRS       12 requirements, 4 corner cases
WBS       1 Epic, 5 Stories (JIRA-ready)
Skill     condensed rules for future prompts
Maturity 100%  -- ready to build
```

Why it matters

Everyone builds from one clear SRS -- no guessing what the client meant.

PART 2 OF 3

Development



We build the change and run a 7-agent review(AI) -- every fix is linked back to the SRS.

/cr	/review	/bug
Change Requests	Code Review	Bug Fixes

Confidential

Change Request Engineer

Overview: The [CR] tag hires the Change Request Engineer. It implements a client change end-to-end across every layer -- Entity, Service, Controller, Template -- following the app's existing patterns.

How It Works

1. Loads the project skill file -- the app's structure and rules
2. Finds the layers to change: Entity, Service, Controller, Template
3. Searches for existing patterns first, then follows the same style
4. Implements the change across all affected layers in one pass
5. Validates against the skill file's acceptance checklist

Result: A complete change across every layer, matching app patterns

Key Benefits

- The project skill file helps it find the right code fast
- Pattern-first: the change matches the rest of the codebase
- Every CR specific acceptance(BASIC) item is checked off before it's done
- Updates every layer -- Entity to Template -- in one pass
- No mixed-up work -- one focused change per prompt

[CR]

Change Request -- How It Works

ex

/cr

One prompt implements a client change across every layer of the app -- here is what that looks like.

What the CR Engineer Does

1 You describe the change
In plain words -- what the client wants, with the key details.

2 It loads the project rules
The skill file tells it the app's structure and patterns.

3 It updates every layer
Entity, then Service, then Controller, then Template -- in order.

4 It checks the result
Validates against the acceptance checklist before finishing.

Example -- add a new field

You type:

```
/cr [CR2] Add a "ceiling_height"
   field, shown only for NEW
   installations
```

produces ▾

```
Change applied across 4 layers:
+ Entity      new field added
+ Service     validation rule
+ Controller  save + load logic
+ Template    shows for NEW only
```

Why it matters

The whole change lands in one pass -- nothing half-built or forgotten.

[Review]

Review(AI) Commander (7 Agents)

3/5

/review

Overview: The [Review] tag hires the Review Commander. Seven specialist agents review the code at once and produce a saved report with a clear verdict and prioritized fixes.

How It Works

1. Picks the mode: a PR, a branch, staged changes, or a diff file
2. Looks at exactly what code has changed
3. 7 agents check its INSTRUCTIONS for itself in parallel : Quality, Security, Performance, Tests, Docs, Errors, Config
4. Each finding is marked Critical, High, Medium, or Low
5. Combined into one verdict: Approve, Approve with Notes, or Needs Changes

Result: A saved CODE-REVIEW report with the verdict and fixes

Key Benefits

- Checking from 7 angles catches issues one reviewer would miss
- A dedicated agent hunts security risks (injection, auth, data leaks)
- A dedicated agent flags slow queries and performance problems
- Every review is saved as a report, so the team has a clear history
- Works at any stage -- PR, branch, or before you commit

[Review]

Code Review(AI) -- How It Works

ex /review

One prompt runs a full 7-agent code review and returns a clear verdict -- here is what that looks like.

What the Review Commander Does

1. **You point it at the code**
   A pull request, a branch, or your staged changes.

2. **7 agents review at once**
   Quality, Security, Performance, Tests, Docs, Errors, Config.

3. **It rates every finding**
   Each issue is marked Critical, High, Medium, or Low.

4. **It gives one verdict**
   Approve, Approve with Notes, or Needs Changes -- saved to a report.

Example -- review a branch

You type:

```
/review branch feature/loyalty-points base feat/cr2-phase1
```

produces ▾

CODE-REVIEW-loyalty-points.md

```
Security     1 High   -- fix first
Performance  2 Medium
Quality      3 Low
Verdict: APPROVE WITH COMMENTS
```

Why it matters

Problems are caught before merge -- with a clear list of what to fix first.

[Bug]

Bug Fixer

Bug

/bug

Overview: The [Bug] tag hires the Bug Fixer. It searches the code to find the real cause of a defect, isolates it, and fixes it -- search-first, so it changes only what's needed.

How It Works

1. You describe the bug -- what is wrong and where you saw it
2. It searches the code (grep-first) to find where it happens
3. It isolates the real cause -- not just the symptom
4. It makes the smallest safe fix, matching the app's style
5. It explains the cause and what was changed

Result: A targeted fix, with a short note on the root cause

Key Benefits

- Search-first: finds the real cause fast, not just the symptom
- Makes the smallest safe change -- low risk of side effects
- Follows the app's existing patterns, so the fix fits in cleanly
- Explains the root cause, so the team learns from each fix
- Works with a project tag for extra context (e.g. [Bug][CR2-Phase2])

[Bug]

Bug Fixer -- How It Works

ex

/bug

One prompt finds the cause of a defect and fixes it -- here is what that looks like.

What the Bug Fixer Does

1. **You report the bug**
   In plain words -- what is wrong and where you saw it.

2. **It searches the code**
   Grep-first: finds where the problem actually happens.

3. **It finds the real cause**
   Isolates the root cause, not just the visible symptom.

4. **It makes a safe fix**
   The smallest change needed, matching the app's style.

Example -- fix a defect

You type:

```
/bug [CR2-Phase2] Order confirmation email not
    sent when payment method is
    bank transfer
```

produces ▾

```
Root cause found:
  email step skipped for bank
  transfer orders
+ 1 file fixed (send-email path)
Fix applied -- email now sends
```

Why it matters

The real cause is fixed once -- not patched over the symptom.

PART 3 OF 3

Testing



BAT reports, unit tests, and Playwright automation -- we check every change before it goes live.

/bat

Ready for QA

/unit-test

Code-Level Tests

Playwright

Full App Tests

Confidential

[BAT]

BAT Generator Agent

/bat

Overview: The [BAT] tag hires the BAT Generator. It reads a code change (branch, PR, or staged), works out what was affected, and writes a ready-to-use acceptance-test report for the QA team -- no manual test writing.

How It Works

1. Reads the git diff for a branch, PR, or staged changes
2. Finds the affected routes, controllers, templates, and services
3. Groups the test cases by module and feature
4. Writes Positive, Negative, and Impact-Area test cases
5. Saves a BAT report (Excel) into docs/BAT/

Result: BAT_[Module]_[Ticket] report in docs/BAT/, ready for QA

Key Benefits

- QA gets a ready acceptance-test file the moment coding is done
- Covers only what changed -- no extra, unrelated test cases
- Positive, Negative, and Impact-Area cases for each module
- Excel output opens straight in QA tools and spreadsheets
- Saves hours of manual test writing -- done from one prompt

BAT Generator -- How It Works

One prompt turns a code change into a full acceptance-test report -- here is what that looks like.

What the BAT Generator Does

1 You give it a change
A branch, a PR, or your staged edits -- nothing else to attach.

2 It reads the diff
Finds the affected screens, controllers, and services on its own.

3 It writes the test cases
Positive (works), Negative (fails safely), and Impact-Area checks.

4 It saves a report
An Excel file in docs/BAT/, grouped by module -- ready for QA.

Example -- generate BAT for a branch

You type:

```
/bat branch feat/P1709-733-posdata base feat/cr1.2
```

produces ▾

docs/BAT/BAT_PosData_P1709-733.csv

Module	Case	Expected result
PosData	Positive	Saves and shows the total
PosData	Negative	Blocks an empty amount
PosData	Impact	Order list still loads

Why it matters
QA starts testing the same day -- no one writes test cases by hand.

[Unit Test]

Test Engineer Agent

5/5

/unit-test

Overview: The [Unit Test] tag hires the Test Engineer. It compares your feature branch to its base, finds the testable code, writes unit / integration tests, and runs them -- so you know the code works right away.

How It Works

1. Compares the feature branch against its base branch
2. Finds the changed PHP files that can be tested
3. Writes clear tests using the Arrange-Act-Assert pattern
4. Uses data providers to cover edge cases in one test
5. Runs the tests (unit or integration) and reports pass/fail

Result: Test files plus a pass/fail run summary

Key Benefits

- Diff-driven: only tests what changed -- no wasted effort
- Arrange-Act-Assert tests are easy to read and maintain
- Data providers catch edge cases developers often miss
- Runs immediately and shows a clear pass/fail result
- Supports both pure unit tests and full integration tests

[Unit Test]

Test Engineer -- How It Works

ex /unit-test

One prompt writes and runs the tests for your latest change -- here is what that looks like.

What the Test Engineer Does

1. **You name the branches**
   Your feature branch and the base it started from.

2. **It finds what to test**
   Compares the two and picks the changed code that can be tested.

3. **It writes the tests**
   Clear Arrange-Act-Assert tests, with edge cases covered.

4. **It runs them for you**
   Runs unit or integration tests and shows a pass/fail summary.

Example -- test a feature branch

You type:

```
/unit-test branch feat/1709632-barcode
          base cr4-phase1
```

produces ▾

tests/.../BarcodeServiceTest.php

```
+ scans a valid barcode
+ rejects an unknown barcode
PASS  2 passed  (0 failed)
```

Why it matters

You confirm the new code works before it ever leaves your machine.

[Playwright]

Test Automation with Playwright

E2E

playwright

End-to-end testing that opens the real app in a browser and checks it behaves correctly for the user.

Overview

Playwright is the tool we use for end-to-end (E2E) testing -- it drives a real browser the way a user would.

Where the other tests check the code, Playwright checks the whole screen:

- **It opens the app in a browser (Chrome, Firefox, Safari)**
- **It clicks, types, and navigates like a real customer**
- **It confirms each screen behaves correctly, end to end**
- **It runs on its own on every code change**

Three AI agents -- Planner, Generator, Healer -- build and maintain these tests (next slide).

Key Benefits

- Tests on all major browsers at once
- Waits for the page automatically -- no guessing with timers
- Shows exactly what happened when a test fails, with screenshots
- Runs fast enough to check every single code change
- Works together with unit tests -- one checks the logic, the other checks the screen

[Playwright]

Playwright: Plan -> Generate -> Heal

AI agents

Three AI agents work together to build and maintain the tests -- from a plan to a passing suite.

1. Planner

Plans the test cases

Reads the spec, requirement, or screen flow and lists every case to test -- positive and negative.

Example -- login plan

```
+ Successful login
- Empty email / password
- Wrong password
- Unregistered email
```

2. Generator

Writes the test code

Turns each planned case into a real, runnable Playwright test file.

Example -- 'empty email' test

```
fill(password, 'Admin@12345')
click('Login')
expect('email is required')
```

3. Healer

Fixes broken tests

Runs the tests and repairs any that fail, so the whole suite passes with no test errors.

Example -- fixed flaky test

```
// email blocked after 5 tries
- 'not-an-email'
+ 'not-an-email-' + Date.now()
```

Confirming a Feature During Development

After building a feature, run its tests to confirm it works -- before moving on.

```
$ npx playwright test login
    run just this feature's tests in CLI
$ npx playwright test login --ui
    watch it run, step by step in browser
```

[MrMax]

MrMax — AI Planning & Documentation Automation

AI agents

Achievements of the Agentic Orchestrator — concrete impact of AI initiatives.

Task	Before	After (with AI)	Time Saved
Requirement Analysis	*4 – 6 hr*	**45 – 60 min**	**~85%**
SRS	*3 – 4 hr*	**20 – 30 min**	**~87%**
WBS	*2 – 3 hr*	**30 min**	**~44%**
BAT Excel	*2 – 3 hr*	**10 – 15 min**	**~80%**
PR Notes	*30 – 45 min*	**5 min**	**~88%**

NEXT TARGET

More Work Volume Next Fiscal Year



Using the same process -- Requirement Analysis, Development, Testing -- to handle more work.

More CRs

Handle More Requests

Faster SRS

Quicker Planning

Full Coverage

Complete Testing

FY Growth

More Work, Same Quality

Confidential

Current scenario

Three AI agents work together to build and maintain the tests -- from a plan to a passing suite.

1. What's going RIGHT

1. Quick Response
2. Feature verification
3. Effective Feature Demo meeting

2. Improvement Points

1. Structured requirements
2. Monthly Review Meeting

3. Risks

F1. Frequent CR during development

Thank You